a repobe keres doksit nezd at es mondd meg mit rakjak meg an?

ZÁRÓDOLGOZAT

Készítették:

Glonczi Tamás – Kulcsár Dénes

Konzulens:

Horváth István

Miskolc

2026.

Miskolci SZC Kandó Kálmán Informatikai Technikum

Miskolci Szakképzési Centrum

SZOFTVERFEJLESZTŐ- ÉS TESZTELŐ SZAK

Itterem

Online éttermi rendelő program

Glonczi Tamás – Kulcsár Dénes

2025-2026

Tartalom

Bevezetés 1
Programozási nyelvek és technológiák 2
Frontend 2
HTML (HyperText Markup Language) 2
CSS (Cascading Style Sheets) 2
JavaScript 2
Vue.js 2
Tailwind CSS keretrendszer 3
Összefoglaló 3
Étlap oldal 4
A Kosár (Cart) működése 5
JWT (JSON Web Token) Hitelesítés – Fejlesztői Dokumentáció 7
2. Miért jó ez nekünk? (A JWT előnyei) 9
3. Miért volt rosszabb az, amit ezelőtt használtunk? 10
MySQL adatbázis (Database) 11
Mi az SQL? (Strukturált lekérdező nyelv) 12
PHPMYADMIN és a XAMPP 12
Backend 14
Web API, ASP.NET CORE 14
MySQL és Entity Framework Core (Database-First) 14
A backend feladata 14
Trello 16
Melléklet 17
Források 18

Bevezetés

Sok kis- és középvállalkozó, kis cégek, vállalatok nem tudnak maguknak megengedni egy komplex, nagy számítási teljesítményt igénylő online rendelőfelület-fejlesztést, erre a problémára nyújt megoldást a mi szoftverünk.

A mai világban azt látjuk, hogy az online leadott rendelések a legegyszerűbbek, legelterjedtebbek és nem utolsósorban a legmegbízhatóbbak. Mindenki találkozott már olyan hellyel, ahol telefonon keresztül kellett a rendelést megadni, és tudjuk: az ember hibázik, rosszul hallja, rosszul írja fel a rendelésünket. A mi szoftverünk ezt a hibafaktort küszöbölné ki a kényelmi előnyei mellett.

A programunk könnyen futtatható, könnyen megtanulható, kis számításigényű, így a kisvállalkozások is könnyen tudják adaptálni a mindennapi munkafolyamatokba, teljesen a megrendeléstől az étel átvételéig.

Ebben a dokumentációban szó esik ennek a programnak a fejlesztési momentumairól.

Programozási nyelvek és technológiák

Programunk backend részét .NET ökoszisztémában csináltuk meg, ez egy nagyon modern, biztonságos és gyors környezet. A felhasználói felület (frontend) fejlesztéséhez modern webes technológiákat használunk: HTML, CSS, JavaScript, valamint a Vue.js keretrendszert a Tailwind CSS stíluskerettel.

Frontend

HTML (HyperText Markup Language)

A HTML biztosítja a weboldal szerkezeti felépítését. Meghatározza az oldalon megjelenő elemek – például gombok, űrlapok, listák és szövegek – hierarchiáját és jelentését.

A projektben a HTML használata azért kulcsfontosságú, mert egy jól strukturált, szemantikusan helyes felépítés javítja a felhasználói élményt, a hozzáférhetőséget és a keresőoptimalizálást. Ez különösen fontos egy olyan rendszer esetében, amelyet különböző eszközökön (mobil, tablet, asztali gép) is használni kell.

CSS (Cascading Style Sheets)

A CSS felelős az alkalmazás megjelenéséért: színek, betűtípusok, elrendezés, animációk és reszponzív viselkedés kialakításáért.

A projekt szempontjából a CSS biztosítja, hogy a rendelési felület és az éttermi adminisztrációs felületek egységes, modern és könnyen átlátható kinézetet kapjanak. A reszponzív tervezésnek köszönhetően a rendszer telefonon és számítógépen is jól használható marad, ami alapkövetelmény egy online rendelési megoldásnál.

JavaScript

A JavaScript a webalkalmazás dinamikus működéséért felel. Segítségével valósítható meg az interakció a felhasználó és a rendszer között, például rendelés leadása, űrlapellenőrzés, értesítések megjelenítése és az adatok valós idejű frissítése.

A projektben a JavaScript biztosítja, hogy a felhasználói élmény gyors és folyamatos legyen, oldal újratöltése nélkül. Ez különösen fontos a rendelési folyamat során, ahol a felhasználó azonnali visszajelzést vár el a műveleteire.

Vue.js

A Vue.js egy nyílt forráskódú, progresszív JavaScript-keretrendszer, amelyet Evan You alkotott meg, és elsősorban modern felhasználói felületek (UI), illetve egyoldalas webalkalmazások (SPA) építésére szolgál. Legfőbb vonzereje a fokozatos bevezethetőség: a fejlesztők dönthetnek úgy, hogy csak egy-egy interaktív komponenshez használják egy meglévő weboldalon, de akár egy teljes, komplex alkalmazás alapját is képezheti.

Működése a Reacthoz hasonlóan komponensalapú és a háttérben virtuális DOM-ot használ a teljesítmény optimalizálására. A Vue azonban egy HTML-alapú sablonszintaxist (template syntax) alkalmaz, ami a legtöbb fejlesztő számára rendkívül intuitívvá és könnyen tanulhatóvá teszi. Kiemelkedő a reaktív adatkötési rendszere, amely automatikusan szinkronban tartja a logikát (adatmodellt) és a felületet. A Vue.js emellett nagyon könnyűsúlyú, rugalmas, és az iparág egyik legalaposabb dokumentációjával rendelkezik, így kezdő és tapasztalt fejlesztők számára is ideális választás.

Tailwind CSS keretrendszer

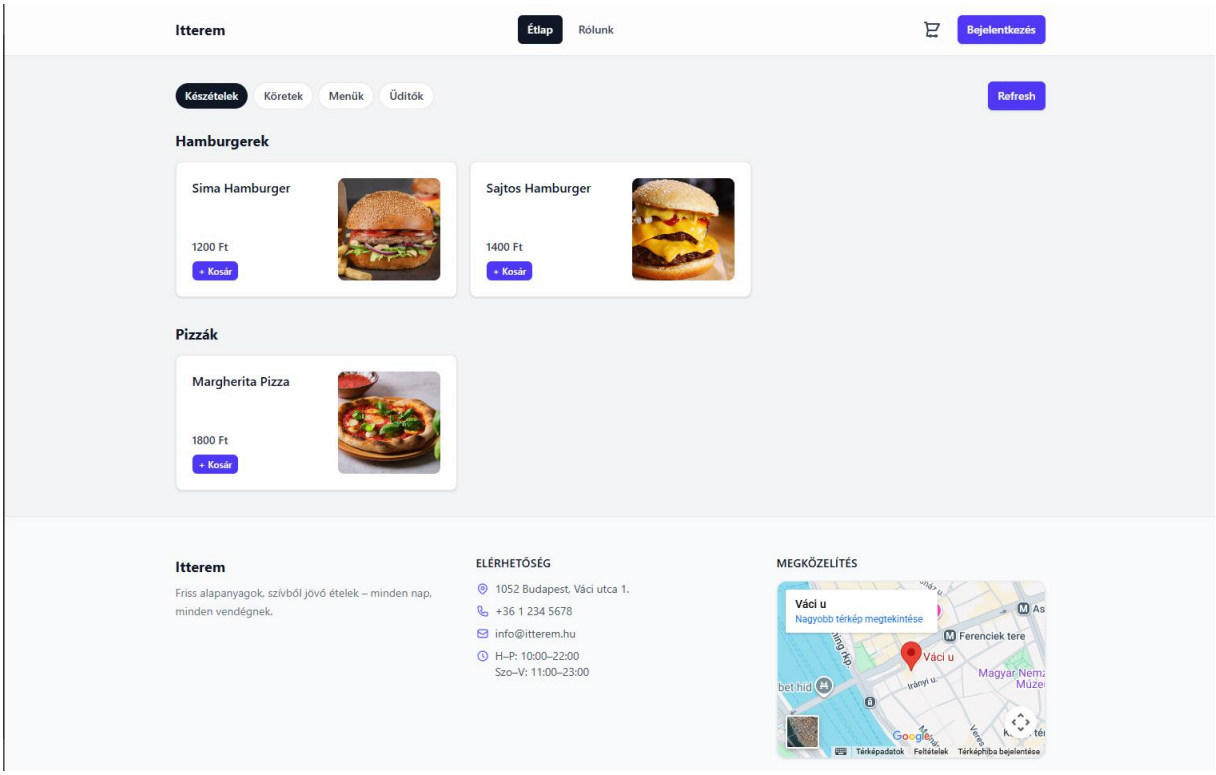
A Tailwind CSS egy modern, úgynevezett „utility-first" (segédosztály-központú) stíluskeretrendszer, amely gyökeresen eltér a hagyományos, komponensalapú megoldásoktól (mint például a Bulma vagy a Bootstrap). Ahelyett, hogy előre megformázott elemeket – például kész gombokat vagy kártyákat – adna a kezünkbe, alacsony szintű, egy-egy konkrét stílustulajdonságot (színek, margók, árnyékok, méretezés) leíró osztályokat kínál. Ezek kombinálásával a fejlesztők közvetlenül a HTML-kódban építhetnek fel teljesen egyedi, reszponzív felületeket.

Ez a megközelítés rendkívüli szabadságot biztosít, hiszen a design megvalósításához alig kell egyedi CSS fájlokat írni, felgyorsítva ezzel a fejlesztési folyamatot. A Tailwind beépített JIT (Just-In-Time) fordítója ráadásul gondoskodik arról, hogy a végső projektbe csak és kizárólag a ténylegesen felhasznált CSS-osztályok kerüljenek bele. Ez extrém kicsi fájlméretet és villámgyors oldalbetöltést eredményez. A keretrendszer moduláris, és kiválóan integrálható a modern JavaScript-technológiákkal, így a Vue.js-szel vagy a Reacttal is zökkenőmentesen együttműködik.

Összefoglaló

A HTML, JavaScript, Vue.js és Tailwind CSS ötvözésével egy olyan moduláris, könnyen karbantartható és biztonságosan bővíthető frontend rendszert hoztunk létre, amely a modern webes sztenderdeknek minden tekintetben megfelel, és maximálisan támogatja az üzleti logikát.

Étlap oldal



Az Étlap oldal megvalósítása

Az étlap oldalt (MenuView.vue) úgy építettem fel, hogy a lehető leggyorsabb és legátláthatóbb legyen a felhasználók számára, így a kategóriákat, ételeket, köreteket és italokat egyszerre, a Promise.allSettled() metódus segítségével töltöm be. Ezt azért csináltam így, hogy a hálózati kérések ne egymásra várjanak, ezáltal drasztikusan csökkentve az oldalindulás idejét. Ráadásul ezzel a megoldással elértem, hogy ha az egyik lista (például az üdítők) lekérdezése hibára fut, a felület többi része attól még zavartalanul megjelenik és használható marad, minden ételtípushoz saját töltés- (loading) és hibajelzőt (errors) is társítottam a pontos visszajelzés érdekében. A készételeket nem egy ömlesztett, végtelen listában jelenítem meg, hanem egy számított tulajdonság (computed property) segítségével a lekérdezett kategóriák (pl. levesek, pizzák) alá csoportosítom őket, a besorolatlan elemeket pedig egy "Egyéb" kategóriába gyűjtöm. Azért döntöttem emellett, hogy a vásárló sokkal könnyebben átlássa a kínálatot.

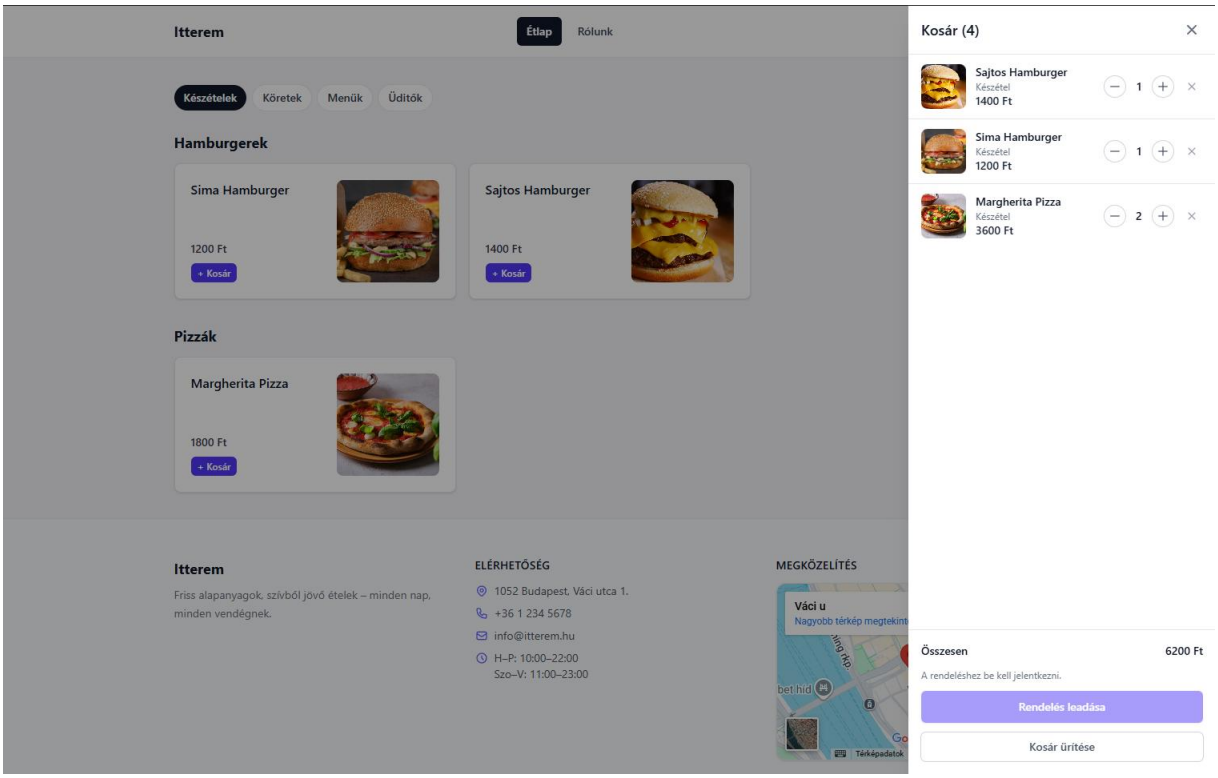
Amikor a felhasználó rákattint egy étel kártyájára, a részletes oldalra (MenuItemPage.vue) lépéskor nem indítok újabb lekérdezést a szerver felé, hanem az adott termék már meglévő adatait elmentem a böngésző sessionStorage-ébe, és az új oldal onnan olvassa ki. Ezt azért csináltam így, hogy az oldalváltás villámgyors és zökkenőmentes legyen, továbbá egy véletlen oldalfrissítés (F5) esetén is megmaradjanak az adatok, felesleges szerverterhelés nélkül. A gördülékenyebb rendelés érdekében a kártyákra tettem egy közvetlen "Kosárba" gombot, így nem kell feltétlenül belépni a részletes nézetbe a vásárláshoz. A "Menük" fülön pedig (ahol például egy "Sajtos menü" szerepel) egy külön formázó logikát írtam (getMenuMeta, buildMenuBreakdown), amivel a rendszer felismeri és szövegesen összefűzi a menühöz tartozó

elemeket (készétel + köret + üdítő). Ezzel az volt a célom, hogy a felhasználó mind az étlapon, mind a részletes oldalon egyértelműen lássa a menü pontos tartalmát, mielőtt kosárba tenné.

A Kosár (Cart) működése

A kosár rendszerét egy saját, központosított állapotkezelővel (useCart.js) oldottam meg, így az egész alkalmazásban – bármelyik oldalon is járjon a felhasználó – egységesen és azonnal elérhető a tartalma, anélkül, hogy bonyolult külső könyvtárakat kellett volna behoznom. A kosár elemeit egy modul-szintű reaktív változóban tárolom, ami "singletonként" viselkedik, vagyis minden komponens pontosan ugyanazt a kosarat látja és módosítja.

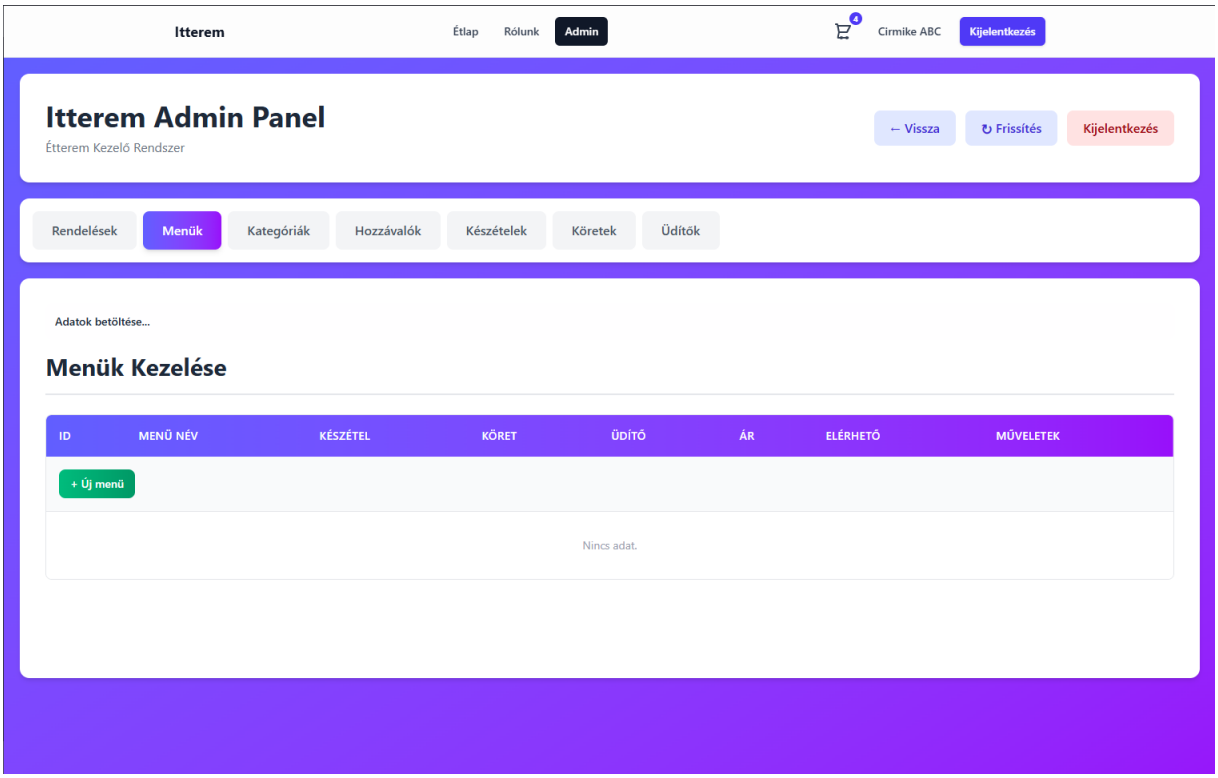
A felhasználói élmény érdekében a kosár tartalmát minden egyes módosításkor (például hozzáadáskor vagy mennyiségnöveléskor) automatikusan elmentem a böngésző localStorage memóriájába. Ezt azért csináltam így, hogy egy véletlen oldalfrissítés, vagy akár a böngésző bezárása után is hiánytalanul megmaradjon a vásárló korábbi rendelése. Magát a kosár felületét (CartDrawer.vue) nem egy külön oldalként, hanem egy oldalról becsúszó panelként valósítottam meg, aminek köszönhetően a vásárló bármikor ránézhet a tételekre és a végösszegre anélkül, hogy el kellene hagynia az étlapot. Amikor a felhasználó a "Rendelés leadása" gombra kattint, a rendszer elsőként ellenőrzi a bejelentkezési státuszát. Ha minden rendben van, a frontend-specifikus adatokat átalakítom a backend API által elvárt formátumra (például a megfelelő keszeteIId vagy uditoId kulcsokra), majd elküldöm a kérést. Siker esetén a kosár azonnal és automatikusan kiürül, a felhasználó pedig egyértelmű visszajelzést kap a rendelés állapotáról.



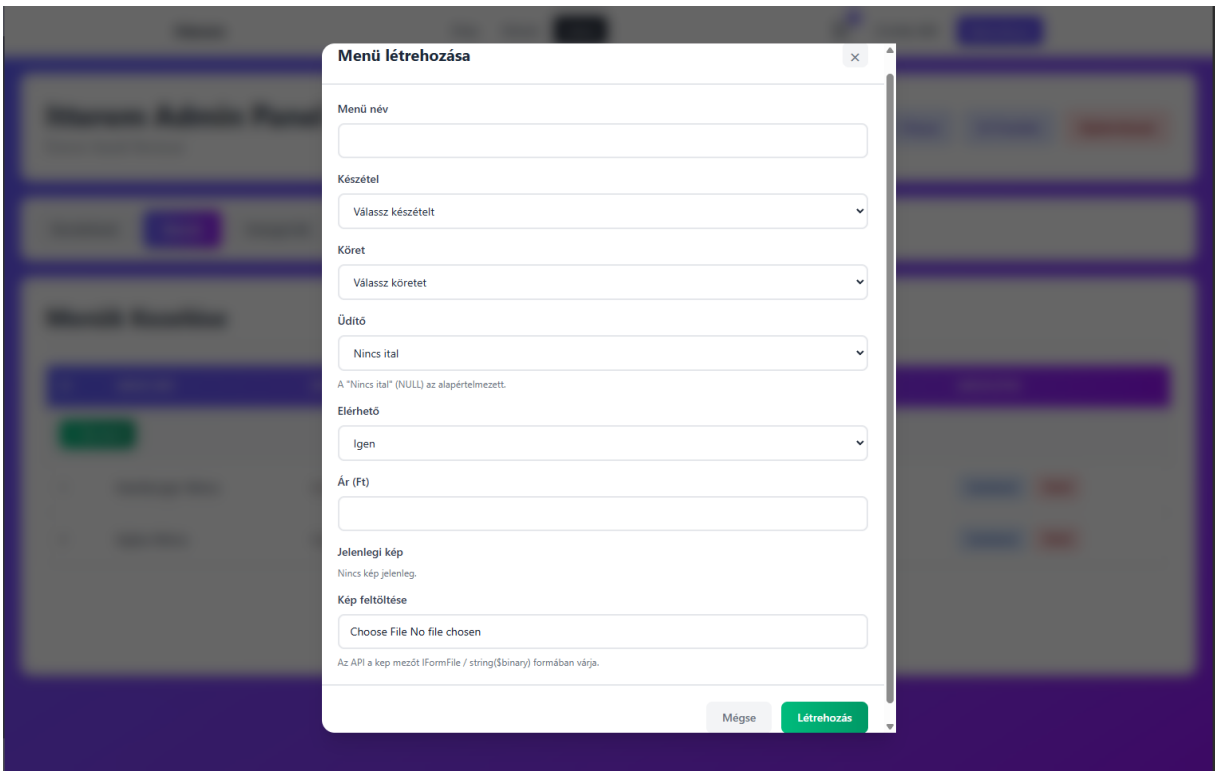
Admin Panel

Az adminisztrációs felületet (AdminDashboard.vue) úgy terveztem meg, hogy egyetlen központi helyről lehessen kezelni az étterem teljes kínálatát, a hozzávalókat és a beérkező rendeléseket. Mivel nagyon sokféle adatot (készételek, köretek, üdítők, kategóriák, menük) kell kilistázni, létrehozni és szerkeszteni, nem akartam minden egyes entitásnak külön táblázatot és űrlapot kódolni. Ehelyett készítettem egy univerzális táblázat (AdminTable.vue) és egy dinamikus szerkesztő ablak (AdminEditModal.vue) komponenst. Ezt azért csináltam így, hogy a kód teljesen ismétlődésmentes (DRY) és könnyen skálázható legyen; a különböző adatkörök megjelenését és működését csupán egyedi konfigurációs objektumokkal (oszlopok definíciói, űrlapmezők típusa, API végpontok) vezérlem a fő komponensből. Ha a jövőben bejön egy új kategória, mondjuk a "Desszertek", pillanatok alatt be tudom kötni a rendszerbe új frontend elemek fejlesztése nélkül.

A bonyolultabb adatátalakításokat, az űrlapok validációját és az adatok formázását (például az árak ezres tagolását vagy az elérhetőségi státuszok logikai konvertálását) tudatosan kiszerveztem egy különálló admin-helpers.js fájlba. Ezzel az volt a célom, hogy a Vue komponensek logikája tiszta és átlátható maradjon, és kizárólag a felület megjelenítésére fókuszáljanak. A biztonságos adatkezelés érdekében az elemek módosítása és törlése nem közvetlenül a listában, hanem mindig felugró ablakokban (modális szerkesztőkben és a ConfirmModal.vue megerősítő panelekben) történik. Ezt a megoldást azért választottam, hogy az adminisztrátor véletlenül se tudjon egyetlen rossz kattintással véglegesen kitörölni egy fontos ételt vagy egy futó rendelést, garantálva a biztonságos és hibamentes munkavégzést.



Admin Panel Szerkesztés/Létrehozás Modal



JWT (JSON Web Token) Hitelesítés – Fejlesztői Dokumentáció

Ebben a részben részletesen bemutatjuk, hogy a projektünkben hogyan alakítottuk ki a felhasználók hitelesítését és jogosultságkezelését JWT (JSON Web Token) segítségével. Igyekeztünk egy robusztus, modern és biztonságos rendszert építeni, ezért fontos

megértenünk, miért pont emellett a technológia mellett döntöttünk, hogyan raktuk össze, és miért váltottuk le vele a korábbi megoldásunkat.

1. Hogyan működik együtt a Frontend és a Backend a JWT-vel?

A rendszerünk architektúráját úgy terveztük meg, hogy a Frontend (Vue.js) és a Backend (ASP.NET Core) teljesen lazán kapcsolódjon egymáshoz. A köztük lévő biztonságos kommunikációt a JWT garantálja.

Ezt a folyamatot a következőképpen alakítottuk ki:

A Backend oldal (C# / ASP.NET Core) szerepe

Amikor a felhasználó beírja az email címét és a jelszavát a felületen, a kérés befut a backendünk LoginController-éhez.

1. **Validáció és Sózás:** Első lépésként ellenőrizzük az emailt az adatbázisban, majd a letárolt jelszó-hash és a "só" (Salt) segítségével (Program.CreateSHA256) verifikáljuk a jelszót.
2. **Token Generálás:** Ha a belépés sikeres, a JwtSecurityToken osztály segítségével legeneráljuk a tokent. Ez egy nagyon fontos lépés volt számunkra: a token úgynevezett **Claim**-ekből épül fel. Beleraktuk a felhasználó azonosítóját (JwtRegisteredClaimNames.Sub), az email címét, a teljes nevét, és ami a legfontosabb, a jogosultsag szintjét (pl. 3-as szint az Admin).
3. **Titkosítás:** Ezt a payload-ot a szerverünk a konfigurációban (appsettings.json) megadott titkos kulccsal (SymmetricSecurityKey) és HmacSha256 algoritmussal digitálisan aláírja.
4. **Végpontok védelme:** A többi végpontunkat (pl. a RendelesekController-ben) [Authorize(Policy = "...")] attribútumokkal láttuk el. Így például egy [Authorize(Policy = "Felhasznalo")] vagy [Authorize(Policy = "Admin")] annotáció esetén a C# backendünk automatikusan és transzparensen ellenőrzi a token aláírását és a jogosultsági szintet. Ha valaki lekéri a saját rendeléseit, a TryGetTokenUserId metódusunkkal biztonságosan kinyerjük a tokenből a felhasználó ID-ját, így a kliens nem tudja meghamisítani, hogy kinek a nevében kér le adatokat.

A Frontend oldal (Vue.js / JavaScript) szerepe

A frontendünkön úgy alakítottuk ki a logikát, hogy az maximálisan kihasználja a JWT adottságait.

1. **Token tárolása:** Sikeres bejelentkezés után a backendtől kapott tokent (és a felhasználói adatokat) a böngésző localStorage-jába mentjük az auth kulcs alá.
2. **Kérések hitelesítése (Bearer token):** Az api.js fájlunkban megírtuk az authHeaders segédfüggvényt. Ez minden egyes védett backend API hívás (fetch) fejlécébe automatikusan beszúrja az Authorization: Bearer <token> értéket. Így a backend mindig tudja, hogy ki küldte a kérést.
3. **Token dekódolása helyben:** A utils.js-ben írtunk egy parseJwt függvényt, ami base64 dekódolással szét tudja szedni a tokent magán a kliensen, anélkül, hogy a backendhez fordulna! Ezzel óriási előnyt nyertünk: a Vue router és a komponenseink azonnal tudják, hogy a felhasználó be van-e jelentkezve, mikor jár le a token (getJwtExpirationMs), és hogy a felhasználó admin-e (isAdmin computed property).
4. **Lejárat kezelése:** Bevezettünk egy token lejárat-figyelőt. Ha a token lejár (isJwtExpired), a rendszer automatikusan elsüt egy itterem:auth-expired eseményt, ami kilépteti a felhasználót, megelőzve ezzel az API oldali "401 Unauthorized" hibaüzeneteket és a rossz felhasználói élményt.

2. Miért jó ez nekünk? (A JWT előnyei)

Azért döntöttünk a JWT teljes körű bevezetése mellett, mert számos architekturális problémánkat megoldotta:

- **Állapotmentesség (Stateless):** A szerverünknek nem kell külön adatbázis lekérdezést futtatnia minden egyes requestnél csak azért, hogy megnézze, ki van bejelentkezve. A token önmagát igazolja az aláírással. Ha a kriptográfiai aláírás stimmel, a backend megbízik a tokenben lévő adatokban (ID, jogosultság).
- **Decentralizált adatkinyerés:** Korábban a frontendnek a betöltés után fel kellett volna hívnia egy végpontot (pl. GET /api/me), hogy megtudja, ki van épp bejelentkezve, és milyen jogai vannak. Mivel a JWT tartalmazza a "Claim"-eket, ezt a Vue.js azonnal kiolvassa memóriából, amivel hálózati kéréseket, ezáltal időt spórolunk.
- **Erős jogosultság-ellenőrzés:** Könnyedén tudtunk Policy-alapú rendszert (pl. Admin_Felhasznalo) építeni az ASP.NET Core-ban a tokenben lévő jogosultsag claim alapján.

3. Miért volt rosszabb az, amit ezelőtt használtunk?

Ha megnézzük a projekt korábbi állapotát és a backend Program.LoggedUsers statikus, szótár alapú memóriás megoldását, egyértelműen látszanak azok a fájdalompontok, amik miatt muszáj volt váltanunk:

1. *Memória-alapú (In-memory) Session elvesztése:* Korábban egy szerver oldali listában (lock (Program.LoggedUsers)) tartottuk nyilván a bejelentkezett felhasználókat a kiadott tokenekkel/azonosítókkal összerendelve. Ez azt jelentette, hogy ha a C# backend bármilyen okból újraindult (pl. egy új verzió deployolása, vagy szerver újraindulás), az *összes aktív felhasználó azonnal ki lett dobva* a rendszerből. A memóriatartalom elveszett, ami rendkívül frusztráló felhasználói élményt okozott. A JWT ezzel szemben a kliensnél van. A szerver újraindulhat akárhányszor, amíg a Titkos Kulcs (Secret Key) nem változik, a felhasználók tokenjei érvényesek maradnak.
2. *Skálázhatósági problémák:* A Program.LoggedUsers típusú globális állapotkezelés egyáltalán nem skálázható. Amíg csak lokálisan futtattuk a fejlesztői gépen a REST API-t, addig működött, de mihelyt több szerverpéldányt szerettünk volna futtatni (Load Balancing), a szerverek nem látták volna egymás memóriáját. A JWT állapotmentessége ezt a gátat azonnal lebontotta.
3. *Felesleges szerver terhelés és lockolás:* Mivel korábban minden kérésnél a memóriában lévő szótárban kellett keresgélnünk a usereket, sőt, a konkurens hozzáférések miatt a lock kulcsszót kellett használnunk a szálbiztosság (thread-safety) miatt, ez komoly teljesítménybeli szűk keresztmetszetet jelentett (bottleneck).

A JWT validáció ezzel szemben egy gyors matematikai művelet, amit a .NET middleware aszinkron és szálbiztos módon végez el.

Összességében kijelenthetjük, hogy a JWT bevezetésével és a frontenddel való szoros integrációjával (a localStorage és az API interceptorok logikájával) egy olyan robusztus, sebességre és skálázhatóságra optimalizált hitelesítési rendszert építettünk, amely mentes a korábbi in-memory session-kezelés gyerekbetegségeitől.

MySQL adatbázis (Database)

A projektünket az adatbázis megtervezésével kezdtük.

Az adatbázis tervezése során az egyik legfontosabb célunk a rugalmas és dinamikus adatkezelés volt. Különös figyelmet fordítottunk a normalizálási szabályokra, hogy elkerüljük a felesleges adatismétléseket.

Ennek érdekében a rendszerben sok helyen a már meglévő adatokat hasznosítjuk újra. Jó példa erre a rendelések kezelése: amikor egy bejelentkezett felhasználó lead egy rendelést, a rendszer nem menti le újra a személyes adatait a rendelés mellé. Ehelyett kizárólag a vásárló egyedi azonosítóját (ID) tároljuk le, amely alapján a felhasználó összes adata bármikor könnyen és frissen lekérdezhető. Ez a megoldás nemcsak helytakarékos, de sokkal biztonságosabb és dinamikusabb is.

Az adatbázis strukturált információk vagy adatok szervezett gyűjteménye, amelyeket jellemzően elektronikusan, számítógépes rendszerben tárolnak. Az adatbázisokat általában adatbázis-kezelő rendszer (DBMS-az a rendszer mely összekapcsolja az adatbázist a végfelhasználóval és kezelni tudják az adatokat benne) vezérli. Az adatokat és az adatbázis-kezelő rendszert, valamint a hozzájuk kapcsolódó alkalmazásokat együttesen adatbázis-rendszernek nevezzük, amit gyakran csak „adatbázis” -ként rövidítünk

A ma használatos leggyakoribb adatbázis-típusokban az adatok jellemzően táblák soraiba és oszlopaiba vannak rendezve, ami megkönnyíti az adatok feldolgozását és lekérdezését. Az adatok így könnyen elérhetők, kezelhetők, módosíthatók, frissíthetők, ellenőrizhetők és rendszerezhetők. A legtöbb adatbázis strukturált lekérdezőnyelvet (SQL) használ az adatok írására és lekérdezésére

Mi az SQL? (Strukturált lekérdező nyelv)

Az SQL egy olyan programozási nyelv, amelyet szinte minden relációs adatbázis** használ az adatok lekérdezésére, kezelésére és meghatározására, valamint a hozzáférés vezérlésére. Az SQL-t az IBM fejlesztette ki az 1970-es években az Oracle jelentős közreműködésével, ami az SQL ANSI-szabvány bevezetéséhez vezetett, majd az SQL számos kiterjesztése követte olyan vállalatoktól, mint az IBM, az Oracle és a Microsoft. Bár az SQL ma is széles körben használatos, új programozási nyelvek is kezdenek megjelenni.

** relációs adatbázis: A relációs adatbázisokban az elemek oszlopokkal és sorokkal rendelkező táblákba vannak rendezve. A relációs adatbázisok technológiája biztosítja a strukturált információk elérésének leghatékonyabb és legrugalmasabb módját. Az 1980-as évektől kezdték meg uralkodásukat a piacon, mai napig is ez a legismertebb és leghasználtabb típusa, ettől független rengeteg másik fajtája is van

(pl.: objektumorientált adatbázisok, elosztott adatbázisok, felhőalapú adatbázisok), de a mai programozási piac és nyelvek a relációs adatbázissal dolgoznak túlnyomó részt.

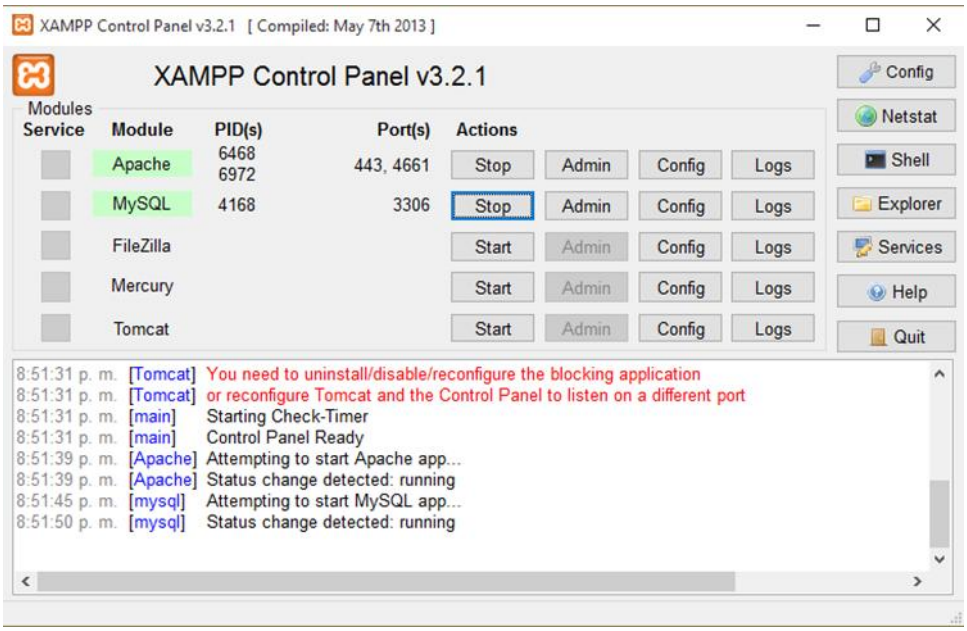
PHPMYADMIN és a XAMPP

Mi a projektünkhöz a PHPMYADMIN nevezetű adatbázis kezelő szoftvert használjuk. A működéséhez szükségünk van a XAMPP nevezetű control panelre.

Mi az a XAMPP?

Mondhatnánk, hogy öt értelmetlen betű egymás mellé téve, de nem az.

A XAMPP egy ingyenes nyílt forráskódú webszerver szoftvercsomag, mely képes értelmezni a PHP-t és főbb alkotóeleme az SQL. Enélkül a felületet nem tudjuk elérni, fontos hogy startoljuk az Apache-t először, utána a MySQL-t, és az Admin gombra kattintva érhetjük el a phpmyadmin felületét.



A PHPMYADMIN a XAMPP-hoz hasonlóan egy nyílt forráskódú, és ingyenes webes adminisztrációs eszköz, mely az egyik legnépszerűbb ilyen alkalmazás, ami képes SQL adatokat kezelni.

Könnyen „hordozható”, mivel webes felületen működik, csak a szerver hozzáférés kell a használatához.

A felületen tudunk új adatbázist létrehozni, táblákat és listákat készíteni, fájlokat importálni és exportálni különböző fájlokként, összetett lekérdezéseket tudunk végrehajtani, adatkapcsolatokat és élő diagrammokat készíteni. SQL parancsokkal is végezhetjük ezeket, de egyszerűbb „kattintgatós” módszerrel is használhatjuk a felületet, ennek köszönhetően gyorsan és hatékonyan tudunk vele dolgozni.

Többféle operációs rendszerrel is együttműködik a felület, így nem kell aggódni, ha nem windows felhasználók vagyunk, rengeteg linux disztribúcióval rendelkezik a program.

Backend

Web API, ASP.NET CORE

A szerveroldali logika megvalósításához a Microsoft által fejlesztett ASP.NET Core keretrendszert használtuk C# nyelven. Ez egy modern és gyors megoldás alkalmazásprogramozási felületek (API-k) építésére. A fejlesztés során figyeltünk arra, hogy a kód könnyen tesztelhető és a későbbiekben egyszerűen bővíthető maradjon (függőséginjektálás használatával). A rendszer biztonságáért, például a bejelentkezések azonosításáért a széles körben elterjedt JWT (JSON Web Token) technológia felel.

MySQL és Entity Framework Core (Database-First)

Az adatok megbízható tárolását a MySQL adatbázis-kezelő végzi. A C# program és az adatbázis közötti kommunikációt az Entity Framework Core (EF Core) segíti. Ez a technológia lehetővé teszi, hogy az adatbázis tábláira úgy hivatkozzunk a kódban, mintha hagyományos C# objektumok lennének. A projekt során úgynevezett adatbázis-első (Database-First) megközelítést alkalmaztunk: először megterveztük a MySQL adatbázis szerkezetét és kapcsolatait, majd a Scaffold-DbContext parancs segítségével ebből automatikusan legeneráltuk a szerveroldali C# modelleket. Ezzel a módszerrel rengeteg ismétlődő, kézi kódolást spóroltunk meg, és minimalizáltuk a hibázás esélyét.

A backend feladata

A backend a hátsó, felhasználónak szempontból láthatatlan, de a programokban fontos és tényleges számításokat végző része.

Feladata a frontend felől érkező adatok gyors feldolgozása, és az eredmények visszajuttatása a frontendnek, de szerver és kliens kapcsolatot is a backend működteti, illetve az adatbázis feldolgozását, és az adatok megfelől védelmi tárolásáért és módosításaiért is a backend felelős.

A backendben végpontokat írunk, ahol az információk cserélődnek, illetve az adatbázisban való módosítást, melyet a frontendtől kap, végzi el, például mentés, törlés, vagy éppen hogyan kerül előhívva az adott információ. Emellett változtatásokat és frissítéseket kell kezelni, illetve törekedni az optimális teljesítményre, hogy minél gyorsabban dolgozza fel egymás között az információt a frontenddel.

Védelem

Jelszavak védelme: Salting és az RNGCryptoServiceProvider

A rendszer a felhasználók jelszavait soha nem tárolja egyszerű, olvasható szövegként az adatbázisban. Ehelyett egy egyirányú titkosítási (hashing) eljárást alkalmazunk, amelyet egy úgynevezett ,,sózási'' (salting) technika tesz biztonságossá.

A sózás lényege, hogy a felhasználó által megadott jelszóhoz a rendszer még a titkosítás előtt hozzáilleszt egy teljesen véletlenszerűen generált, egyedi karaktersorozatot (ez a \textit{Salt}). Ezt a véletlenszerű értéket a .NET beépített, kriptográfiailag biztonságos véletlenszám-generátora, az RNGCryptoServiceProvider állítja elő minden egyes regisztráció vagy jelszómódosítás alkalmával.

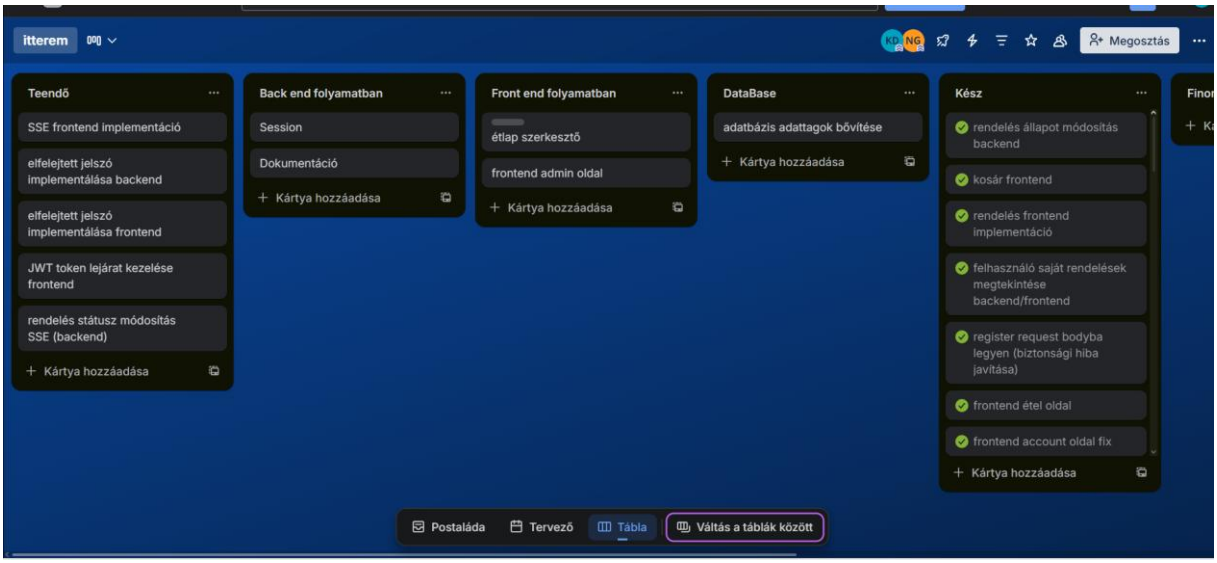
Ennek a módszernek a használata kritikus fontosságú. Ha két felhasználó véletlenül pontosan ugyanazt a jelszót (pl. ,,pizza123'') adná meg, sózás nélkül a titkosított adatbázis-bejegyzésük is hajszálpontosan megegyezne. Egy esetleges adatszivárgás esetén a támadók a gyakori jelszavakat tartalmazó ,,szivárványtáblák'' segítségével azonnal visszafejthetnék ezeket. Mivel azonban a rendszerünk minden felhasználóhoz egyedi Salt értéket generál, és ezt a jelszóval együtt titkosítja, ugyanaz a jelszó két különböző embernél teljesen más titkosított végeredményt fog adni. Ez a megközelítés gyakorlatilag lehetetlenné teszi a szótáralapú támadásokat.

Titkosító algoritmus (SHA-256)

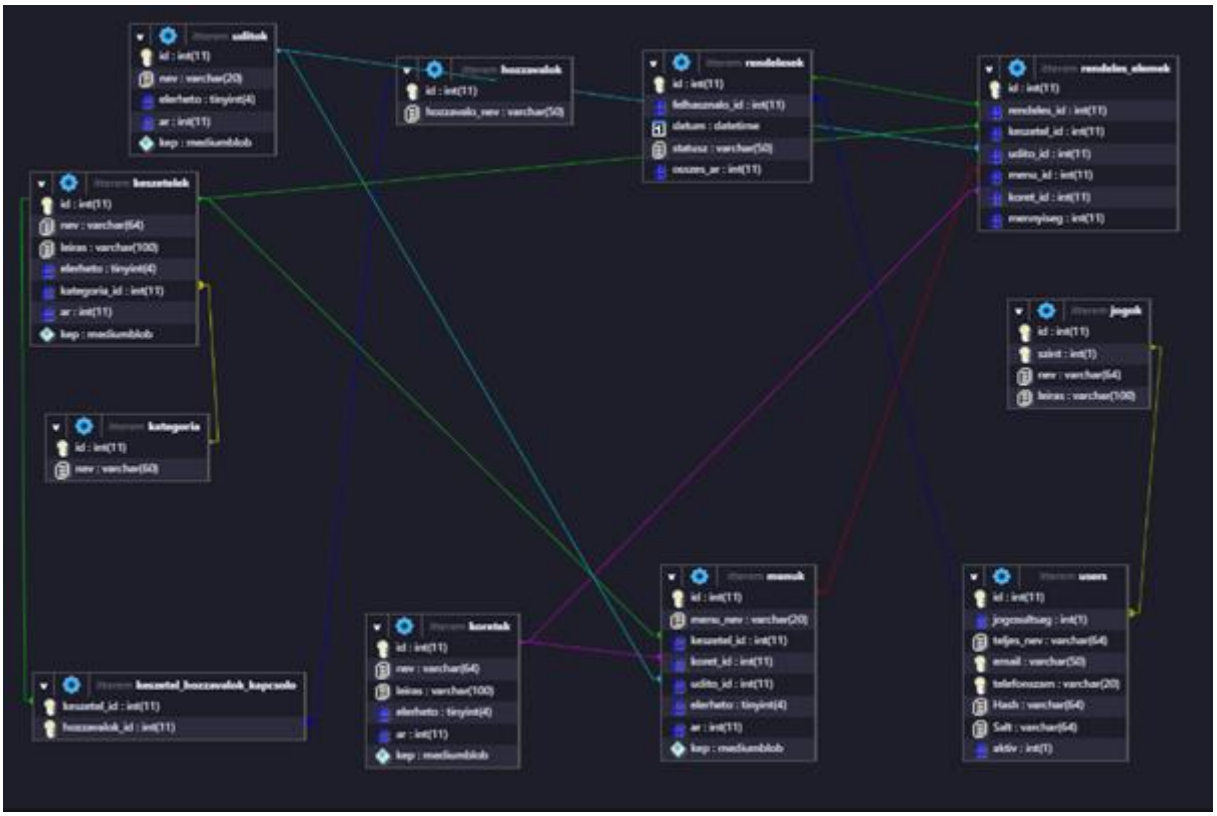
A sóval kiegészített jelszó lenyomatának (hash) elkészítéséhez a széles körben elterjedt SHA-256 (Secure Hash Algorithm 256-bit) algoritmust használjuk. Ez az eljárás a bemeneti (sózott) jelszóból egy fix hosszúságú, 256 bites lenyomatot képez. Mivel az SHA-256 egyirányú függvény, a generált lenyomatból matematikai úton lehetetlen visszafejteni az eredeti jelszót. Bejelentkezéskor a rendszer a megadott jelszót az adatbázisban tárolt egyedi sóval újra titkosítja, és ha az eredmény megegyezik a tárolt lenyomattal, a hitelesítés sikeres.

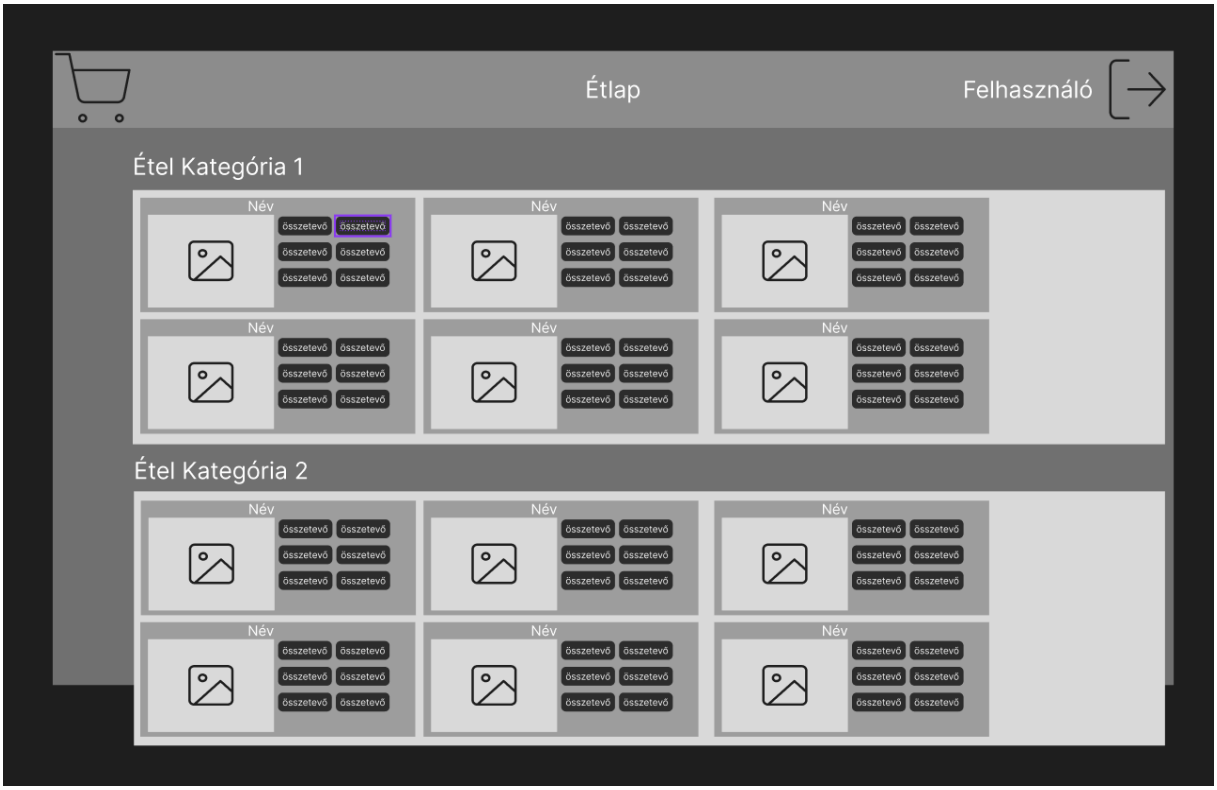
Trello

Trellót használtuk a projektfejlesztés során. Ennek segítségével könnyen tudtuk kezelni az éppen zajló munkafolyamatokat, láttuk, mit kell még csinálnunk, mit mikorra kell megcsinálni. Mivel a kártyákat mindenki tudja mozgatni, így ha valami hiányzik a frontendhez, könnyen tudok hozzáadni egy másik kártyát, amit a backendes lát és meg tud csinálni, így a munka gördülékenyen megy, nincs megakadás.



Melléklet




Étlap
Felhasználó
Étel Kategória 1
Név
összetevő
Étel Kategória 2

Források

https://www.oracle.com/hu/database/what-is-database , letöltve :2026.02.23

https://learn.microsoft.com/en-us/dotnet/csharp/ , letöltve : 2026.02.23

https://hu.wikipedia.org/wiki/Entity_Framework/ , letöltve : 2026.02.23

https://vuejs.org/guide/introduction.html , letöltve : 2026.02.23

https://www.jwt.io/introduction#what-is-json-web-token , letöltve : 2026.02.23

https://tailwindcss.com , letöltve : 2026.02.23